

6.2 Queues

Another fundamental data structure is the *queue*. It is a close “cousin” of the stack, but a queue is a collection of objects that are inserted and removed according to the *first-in, first-out (FIFO)* principle. That is, elements can be inserted at any time, but only the element that has been in the queue the longest can be next removed.

We usually say that elements enter a queue at the back and are removed from the front. A metaphor for this terminology is a line of people waiting to get on an amusement park ride. People waiting for such a ride enter at the back of the line and get on the ride from the front of the line. There are many other applications of queues (see Figure 6.4). Stores, theaters, reservation centers, and other similar services typically process customer requests according to the FIFO principle. A queue would therefore be a logical choice for a data structure to handle calls to a customer service center, or a wait-list at a restaurant. FIFO queues are also used by many computing devices, such as a networked printer, or a Web server responding to requests.

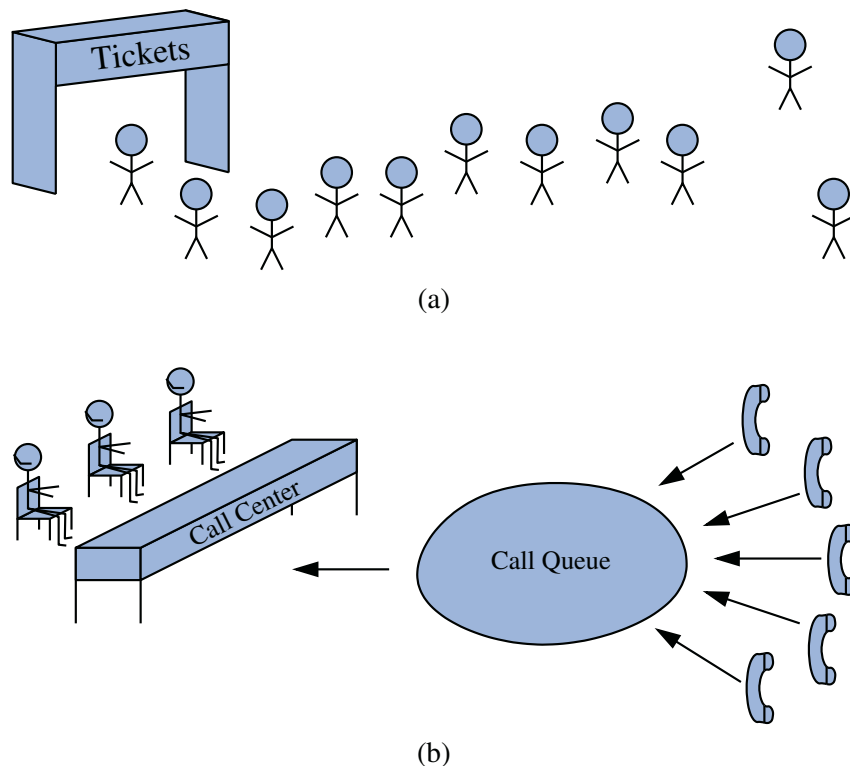


Figure 6.4: Real-world examples of a first-in, first-out queue. (a) People waiting in line to purchase tickets; (b) phone calls being routed to a customer service center.

6.2.1 The Queue Abstract Data Type

Formally, the queue abstract data type defines a collection that keeps objects in a sequence, where element access and deletion are restricted to the *first* element in the queue, and element insertion is restricted to the back of the sequence. This restriction enforces the rule that items are inserted and deleted in a queue according to the first-in, first-out (FIFO) principle. The *queue* abstract data type (ADT) supports the following two update methods:

`enqueue(e)`: Adds element *e* to the back of queue.

`dequeue()`: Removes and returns the first element from the queue (or null if the queue is empty).

The queue ADT also includes the following accessor methods (with first being analogous to the stack's top method):

`first()`: Returns the first element of the queue, without removing it (or null if the queue is empty).

`size()`: Returns the number of elements in the queue.

`isEmpty()`: Returns a boolean indicating whether the queue is empty.

By convention, we assume that elements added to the queue can have arbitrary type and that a newly created queue is empty. We formalize the queue ADT with the Java interface shown in Code Fragment 6.9.

```
1 public interface Queue<E> {  
2     /** Returns the number of elements in the queue. */  
3     int size();  
4     /** Tests whether the queue is empty. */  
5     boolean isEmpty();  
6     /** Inserts an element at the rear of the queue. */  
7     void enqueue(E e);  
8     /** Returns, but does not remove, the first element of the queue (null if empty). */  
9     E first();  
10    /** Removes and returns the first element of the queue (null if empty). */  
11    E dequeue();  
12 }
```

Code Fragment 6.9: A Queue interface defining the queue ADT, with a standard FIFO protocol for insertions and removals.

Example 6.4: The following table shows a series of queue operations and their effects on an initially empty queue Q of integers.

Method	Return Value	first $\leftarrow Q \leftarrow$ last
enqueue(5)	—	(5)
enqueue(3)	—	(5, 3)
size()	2	(5, 3)
dequeue()	5	(3)
isEmpty()	false	(3)
dequeue()	3	()
isEmpty()	true	()
dequeue()	null	()
enqueue(7)	—	(7)
enqueue(9)	—	(7, 9)
first()	7	(7, 9)
enqueue(4)	—	(7, 9, 4)

The java.util.Queue Interface in Java

Java provides a type of queue interface, `java.util.Queue`, which has functionality similar to the traditional queue ADT, given above, but the documentation for the `java.util.Queue` interface does not insist that it support only the FIFO principle. When supporting the FIFO principle, the methods of the `java.util.Queue` interface have the equivalences with the queue ADT shown in Table 6.3.

The `java.util.Queue` interface supports two styles for most operations, which vary in the way that they treat exceptional cases. When a queue is empty, the `remove()` and `element()` methods throw a `NoSuchElementException`, while the corresponding methods `poll()` and `peek()` return **null**. For implementations with a bounded capacity, the `add` method will throw an `IllegalStateException` when full, while the `offer` method ignores the new element and returns **false** to signal that the element was not accepted.

Our Queue ADT	Interface <code>java.util.Queue</code>	
	throws exceptions	returns special value
enqueue(e)	add(e)	offer(e)
dequeue()	remove()	poll()
first()	element()	peek()
size()	size()	
isEmpty()	isEmpty()	

Table 6.3: Methods of the queue ADT and corresponding methods of the interface `java.util.Queue`, when supporting the FIFO principle.

6.2.2 Array-Based Queue Implementation

In Section 6.1.2, we implemented the LIFO semantics of the Stack ADT using an array (albeit, with a fixed capacity), such that every operation executes in constant time. In this section, we will consider how to use an array to efficiently support the FIFO semantics of the Queue ADT.

Let's assume that as elements are inserted into a queue, we store them in an array such that the first element is at index 0, the second element at index 1, and so on. (See Figure 6.5.)

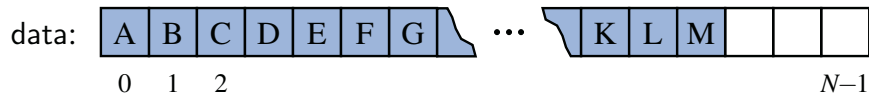


Figure 6.5: Using an array to store elements of a queue, such that the first element inserted, “A”, is at cell 0, the second element inserted, “B”, at cell 1, and so on.

With such a convention, the question is how we should implement the dequeue operation. The element to be removed is stored at index 0 of the array. One strategy is to execute a loop to shift all other elements of the queue one cell to the left, so that the front of the queue is again aligned with cell 0 of the array. Unfortunately, the use of such a loop would result in an $O(n)$ running time for the dequeue method.

We can improve on the above strategy by avoiding the loop entirely. We will replace a dequeued element in the array with a null reference, and maintain an explicit variable f to represent the index of the element that is currently at the front of the queue. Such an algorithm for dequeue would run in $O(1)$ time. After several dequeue operations, this approach might lead to the configuration portrayed in Figure 6.6.

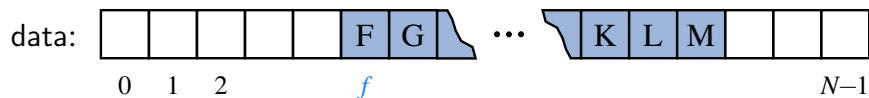


Figure 6.6: Allowing the front of the queue to drift away from index 0. In this representation, index f denotes the location of the front of the queue.

However, there remains a challenge with the revised approach. With an array of capacity N , we should be able to store up to N elements before reaching any exceptional case. If we repeatedly let the front of the queue drift rightward over time, the back of the queue would reach the end of the underlying array even when there are fewer than N elements currently in the queue. We must decide how to store additional elements in such a configuration.

Using an Array Circularly

In developing a robust queue implementation, we allow both the front and back of the queue to drift rightward, with the contents of the queue “wrapping around” the end of an array, as necessary. Assuming that the array has fixed length N , new elements are enqueued toward the “end” of the current queue, progressing from the front to index $N - 1$ and continuing at index 0, then 1. Figure 6.7 illustrates such a queue with first element F and last element R.

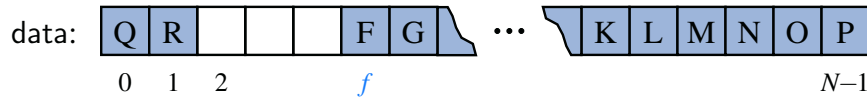


Figure 6.7: Modeling a queue with a circular array that wraps around the end.

Implementing such a circular view is relatively easy with the *modulo* operator, denoted with the symbol `%` in Java. Recall that the modulo operator is computed by taking the remainder after an integral division. For example, 14 divided by 3 has a quotient of 4 with remainder 2, that is, $\frac{14}{3} = 4\frac{2}{3}$. So in Java, `14 / 3` evaluates to the quotient 4, while `14 % 3` evaluates to the remainder 2.

The modulo operator is ideal for treating an array circularly. When we dequeue an element and want to “advance” the front index, we use the arithmetic $f = (f + 1) \% N$. As a concrete example, if we have an array of length 10, and a front index 7, we can advance the front by formally computing $(7 + 1) \% 10$, which is simply 8, as 8 divided by 10 is 0 with a remainder of 8. Similarly, advancing index 8 results in index 9. But when we advance from index 9 (the last one in the array), we compute $(9 + 1) \% 10$, which evaluates to index 0 (as 10 divided by 10 has a remainder of zero).

A Java Queue Implementation

A complete implementation of a queue ADT using an array in circular fashion is presented in Code Fragment 6.10. Internally, the queue class maintains the following three instance variables:

- data:** a reference to the underlying array.
- f:** an integer that represents the index, within array `data`, of the first element of the queue (assuming the queue is not empty).
- sz:** an integer representing the current number of elements stored in the queue (not to be confused with the length of the array).

We allow the user to specify the capacity of the queue as an optional parameter to the constructor.

The implementations of methods `size` and `isEmpty` are trivial, given the `sz` field, and the implementation of `first` is simple, given index `f`. A discussion of update methods `enqueue` and `dequeue` follows the presentation of the code.

```

1  /** Implementation of the queue ADT using a fixed-length array. */
2  public class ArrayQueue<E> implements Queue<E> {
3      // instance variables
4      private E[] data;                // generic array used for storage
5      private int f = 0;                // index of the front element
6      private int sz = 0;                // current number of elements
7
8      // constructors
9      public ArrayQueue() {this(CAPACITY);} // constructs queue with default capacity
10     public ArrayQueue(int capacity) {      // constructs queue with given capacity
11         data = (E[]) new Object[capacity]; // safe cast; compiler may give warning
12     }
13
14     // methods
15     /** Returns the number of elements in the queue. */
16     public int size() { return sz; }
17
18     /** Tests whether the queue is empty. */
19     public boolean isEmpty() { return (sz == 0); }
20
21     /** Inserts an element at the rear of the queue. */
22     public void enqueue(E e) throws IllegalStateException {
23         if (sz == data.length) throw new IllegalStateException("Queue is full");
24         int avail = (f + sz) % data.length; // use modular arithmetic
25         data[avail] = e;
26         sz++;
27     }
28
29     /** Returns, but does not remove, the first element of the queue (null if empty). */
30     public E first() {
31         if (isEmpty()) return null;
32         return data[f];
33     }
34
35     /** Removes and returns the first element of the queue (null if empty). */
36     public E dequeue() {
37         if (isEmpty()) return null;
38         E answer = data[f];
39         data[f] = null; // dereference to help garbage collection
40         f = (f + 1) % data.length;
41         sz--;
42         return answer;
43     }

```

Code Fragment 6.10: Array-based implementation of a queue.

Adding and Removing Elements

The goal of the enqueue method is to add a new element to the back of the queue. We need to determine the proper index at which to place the new element. Although we do not explicitly maintain an instance variable for the back of the queue, we compute the index of the next opening based on the formula:

$$\text{avail} = (f + \text{sz}) \% \text{data.length};$$

Note that we are using the size of the queue as it exists *prior* to the addition of the new element. As a sanity check, for a queue with capacity 10, current size 3, and first element at index 5, its three elements are stored at indices 5, 6, and 7, and the next element should be added at index 8, computed as $(5+3) \% 10$. As a case with wraparound, if the queue has capacity 10, current size 3, and first element at index 8, its three elements are stored at indices 8, 9, and 0, and the next element should be added at index 1, computed as $(8+3) \% 10$.

When the dequeue method is called, the current value of *f* designates the index of the value that is to be removed and returned. We keep a local reference to the element that will be returned, before setting its cell of the array back to **null**, to aid the garbage collector. Then the index *f* is updated to reflect the removal of the first element, and the presumed promotion of the second element to become the new first. In most cases, we simply want to increment the index by one, but because of the possibility of a wraparound configuration, we rely on modular arithmetic, computing $f = (f+1) \% \text{data.length}$, as originally described on page 242.

Analyzing the Efficiency of an Array-Based Queue

Table 6.4 shows the running times of methods in a realization of a queue by an array. As with our array-based stack implementation, each of the queue methods in the array realization executes a constant number of statements involving arithmetic operations, comparisons, and assignments. Thus, each method in this implementation runs in $O(1)$ time.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
first	$O(1)$
enqueue	$O(1)$
dequeue	$O(1)$

Table 6.4: Performance of a queue realized by an array. The space usage is $O(N)$, where N is the size of the array, determined at the time the queue is created, and independent from the number $n < N$ of elements that are actually in the queue.

6.2.3 Implementing a Queue with a Singly Linked List

As we did for the stack ADT, we can easily adapt a singly linked list to implement the queue ADT while supporting worst-case $O(1)$ -time for all operations, and without any artificial limit on the capacity. The natural orientation for a queue is to align the front of the queue with the front of the list, and the back of the queue with the tail of the list, because the only update operation that singly linked lists support at the back end is an insertion. Our Java implementation of a `LinkedList` class is given in Code 6.11.

```
1  /** Realization of a FIFO queue as an adaptation of a SinglyLinkedList. */
2  public class LinkedList<E> implements Queue<E> {
3      private SinglyLinkedList<E> list = new SinglyLinkedList<>(); // an empty list
4      public LinkedList() { } // new queue relies on the initially empty list
5      public int size() { return list.size(); }
6      public boolean isEmpty() { return list.isEmpty(); }
7      public void enqueue(E element) { list.addLast(element); }
8      public E first() { return list.first(); }
9      public E dequeue() { return list.removeFirst(); }
10 }
```

Code Fragment 6.11: Implementation of a Queue using a SinglyLinkedList.

Analyzing the Efficiency of a Linked Queue

Although we had not yet introduced asymptotic analysis when we presented our SinglyLinkedList implementation in Chapter 3, it is clear upon reexamination that each method of that class runs in $O(1)$ worst-case time. Therefore, each method of our LinkedList adaptation also runs in $O(1)$ worst-case time.

We also avoid the need to specify a maximum size for the queue, as was done in the array-based queue implementation. However, this benefit comes with some expense. Because each node stores a next reference, in addition to the element reference, a linked list uses more space per element than a properly sized array of references.

Also, although all methods execute in constant time for both implementations, it seems clear that the operations involving linked lists have a large number of primitive operations per call. For example, adding an element to an array-based queue consists primarily of calculating an index with modular arithmetic, storing the element in the array cell, and incrementing the size counter. For a linked list, an insertion includes the instantiation and initialization of a new node, relinking an existing node to the new node, and incrementing the size counter. In practice, this makes the linked-list method more expensive than the array-based method.

6.2.4 A Circular Queue

In Section 3.3, we implemented a *circularly linked list* class that supports all behaviors of a singly linked list, and an additional `rotate()` method that efficiently moves the first element to the end of the list. We can generalize the Queue interface to define a new CircularQueue interface with such a behavior, as shown in Code Fragment 6.12.

```
1 public interface CircularQueue<E> extends Queue<E> {  
2     /**  
3     * Rotates the front element of the queue to the back of the queue.  
4     * This does nothing if the queue is empty.  
5     */  
6     void rotate();  
7 }
```

Code Fragment 6.12: A Java interface, CircularQueue, that extends the Queue ADT with a new `rotate()` method.

This interface can easily be implemented by adapting the CircularlyLinkedList class of Section 3.3 to produce a new LinkedCircularQueue class. This class has an advantage over the traditional LinkedQueue, because a call to `Q.rotate()` is implemented more efficiently than the combination of calls, `Q.enqueue(Q.dequeue())`, because no nodes are created, destroyed, or relinked by the implementation of a rotate operation on a circularly linked list.

A circular queue is an excellent abstraction for applications in which elements are cyclically arranged, such as for multiplayer, turn-based games, or round-robin scheduling of computing processes. In the remainder of this section, we provide a demonstration of the use of a circular queue.

The Josephus Problem

In the children’s game “hot potato,” a group of n children sit in a circle passing an object, called the “potato,” around the circle. The potato begins with a starting child in the circle, and the children continue passing the potato until a leader rings a bell, at which point the child holding the potato must leave the game after handing the potato to the next child in the circle. After the selected child leaves, the other children close up the circle. This process is then continued until there is only one child remaining, who is declared the winner. If the leader always uses the strategy of ringing the bell so that every k^{th} person is removed from the circle, for some fixed value k , then determining the winner for a given list of children is known as the *Josephus problem* (named after an ancient story with far more severe consequences than in the children’s game).

Solving the Josephus Problem Using a Queue

We can solve the Josephus problem for a collection of n elements using a circular queue, by associating the potato with the element at the front of the queue and storing elements in the queue according to their order around the circle. Thus, passing the potato is equivalent to rotating the first element to the back of the queue. After this process has been performed $k - 1$ times, we remove the front element by dequeuing it from the queue and discarding it. We show a complete Java program for solving the Josephus problem using this approach in Code Fragment 6.13, which describes a solution that runs in $O(nk)$ time. (We can solve this problem faster using techniques beyond the scope of this book.)

```

1  public class Josephus {
2      /** Computes the winner of the Josephus problem using a circular queue. */
3      public static <E> E Josephus(CircularQueue<E> queue, int k) {
4          if (queue.isEmpty()) return null;
5          while (queue.size() > 1) {
6              for (int i=0; i < k-1; i++) // skip past k-1 elements
7                  queue.rotate();
8              E e = queue.dequeue(); // remove the front element from the collection
9              System.out.println("    " + e + " is out");
10         }
11         return queue.dequeue(); // the winner
12     }
13
14     /** Builds a circular queue from an array of objects. */
15     public static <E> CircularQueue<E> buildQueue(E a[ ]) {
16         CircularQueue<E> queue = new LinkedCircularQueue<>();
17         for (int i=0; i<a.length; i++)
18             queue.enqueue(a[i]);
19         return queue;
20     }
21
22     /** Tester method */
23     public static void main(String[ ] args) {
24         String[ ] a1 = {"Alice", "Bob", "Cindy", "Doug", "Ed", "Fred"};
25         String[ ] a2 = {"Gene", "Hope", "Irene", "Jack", "Kim", "Lance"};
26         String[ ] a3 = {"Mike", "Roberto"};
27         System.out.println("First winner is " + Josephus(buildQueue(a1), 3));
28         System.out.println("Second winner is " + Josephus(buildQueue(a2), 10));
29         System.out.println("Third winner is " + Josephus(buildQueue(a3), 7));
30     }
31 }

```

Code Fragment 6.13: A complete Java program for solving the Josephus problem using a circular queue.